

APPLIED AGENTIC AI FOR PMS / TPMS

Applied Agentic AI for PMS / TPMS — Capstone Project

CalendarMate

AI-Powered Productivity Assistant

Company: ServionIQ Solutions

Role: AI PM / TPM

Platform: n8n | Model: GPT-4o-mini

June 2026

Business Case

Executive Summary

ServionIQ Solutions operates at the intersection of professional services and cloud-native software, where its product and technical teams run at relentless pace. PMs and TPMs — the people most responsible for keeping delivery on track — are paradoxically drowning in the very coordination work that slows delivery down. **CalendarMate is an AI-powered, multi-agent productivity assistant built on n8n that reclaims this lost time by automating daily briefings, intelligent scheduling, email triage, and post-meeting follow-up generation.**

The Business Problem

A typical ServionIQ PM or TPM manages 8–14 meetings per day across engineering, product, design, and client teams. Before the first meeting even starts, they spend 30–45 minutes piecing together what is happening — manually cross-referencing 3–5 team calendars, triaging 60+ emails, and hunting for context across Slack, docs, and calendar invites. By the time they sit down to do actual product work, nearly a quarter of their working day is gone.

The direct cost of this inefficiency is significant:

- **2 hours per person per day** consumed by meeting logistics and email triage
- Frequent **double-bookings** because availability checks are done manually across multiple calendars
- **Missed follow-ups** — action items discussed in meetings are rarely tracked systematically, leading to rework and missed commitments
- No **consolidated daily view** — context-switching between calendar, email, Slack, and docs breaks deep-work focus
- For a team of 50 PMs/TPMs, this amounts to **roughly 100 person-hours lost every single day** to administrative overhead instead of high-value product thinking

The Solution: CalendarMate

CalendarMate is a five-agent AI system deployed on n8n that connects directly to Google Calendar and Gmail via OAuth2. It understands natural language requests, fetches real-time data from live APIs, and returns structured, grounded outputs — with strict guardrails to prevent hallucination. The system replaces manual morning triage with an instant AI-generated briefing, eliminates scheduling back-and-forth through conversational event creation, tames the inbox with intelligent categorisation, and turns raw meeting notes into polished follow-up emails and action item lists.

Business Benefits

Benefit 1 — Recovered Productivity

If CalendarMate reduces daily logistics overhead from 2 hours to 30 minutes per PM/TPM, each person recovers **90 minutes of deep-work time per day**. Across a team of 50, that is **75 person-hours recovered daily** — equivalent to adding nearly 10 full-time employees worth of productive capacity without a single new hire.

Benefit 2 — Reduced Scheduling Errors and Meeting Conflicts

Manual availability checks across 3–5 calendars produce regular double-bookings and last-minute reschedules, each of which costs additional time for all participants. CalendarMate's Scheduler Agent checks live availability before proposing any slot and requires explicit confirmation before creating events. **Eliminating even 2 conflicts per PM per week saves approximately 30 minutes of wasted coordination per incident** — compounding to thousands of recovered hours annually across the organisation.

Benefit 3 — Improved Decision Quality and Accountability

Missed follow-ups are a silent organisational cost. When action items are not tracked, decisions made in meetings do not translate to outcomes. The Follow-Up Agent converts any meeting with notes into a structured summary, owner-assigned action item list, and ready-to-send email in seconds. **This directly improves delivery accountability and reduces the rework caused by dropped commitments.**

Key Business Metrics

CalendarMate's impact will be tracked across three primary dimensions:

Metric	Definition	Measurement Method	Target
Time Saved Per User Per Day	Minutes saved on meeting logistics vs pre-deployment baseline	Pre/post user survey + n8n execution logs tracking daily usage volume	≥ 60 min/day saved within 30 days of rollout
Scheduling Accuracy Rate	% of calendar events created at the correct time with correct attendees, with no conflicts	Cross-reference created events vs user-confirmed parameters logged during Python evaluation runs in Google Colab	≥ 95% accuracy
Hallucination Rate	% of AI outputs containing information not present in the source calendar/email data	Manual audit of outputs captured during Python evaluation runs. Automated keyword mismatch check comparing agent output against source input data.	< 2% of pipeline runs
Email Triage Precision	% of emails correctly classified as Action Required vs FYI by the Email Agent	Monthly random sample review by team leads against actual email content	≥ 90% classification accuracy
Daily Active Usage Rate	% of target PMS/TPMs using CalendarMate at least once per working day	n8n execution logs by user	≥ 80% DAU within 60 days of full rollout
Cost Per User Per Month	Total LLM + API cost attributable to one	Token usage logged during Python evaluation	< \$0.50 / user / month

Metric	Definition	Measurement Method	Target
	user's daily CalendarMate usage	in Google Colab × OpenAI pricing	

Return on Investment

At an estimated **\$0.36 per user per month** in LLM costs (15 pipeline runs/day × ~2,800 tokens × GPT-4o-mini pricing), CalendarMate costs less than **\$220 per month** for 50 users. If it recovers even 30 minutes of billable or productive time per PM per day, and those PMs have a fully-loaded cost of \$80/hour, the monthly value delivered is approximately **\$300,000**. The ROI is not marginal — it is transformational at negligible cost.

Business Case Summary

ServionIQ's PMs and TPMs spend 2 hours daily on meeting logistics that AI can handle in seconds. CalendarMate eliminates this overhead through a multi-agent system grounded in live calendar and email data, recovering 75+ person-hours per day across the team, reducing scheduling errors, and improving delivery accountability — all for under \$0.50 per user per month.

Build CalendarMate

Design & Architecture

Orchestration Pattern: Router / Dispatcher

CalendarMate uses a **Router (Dispatcher) orchestration pattern**. This was selected because user requests are fundamentally heterogeneous in nature — a briefing request requires only calendar data, a scheduling request requires calendar reads plus conditional event creation, an email request requires Gmail data exclusively, and a follow-up request requires notes from the user plus recent email context. Forcing all requests through a single sequential pipeline would waste API calls, increase latency, and bloat token usage on irrelevant data.

A Sequential Pipeline was explicitly rejected because it assumes every input follows the same processing steps in the same order — which is false for CalendarMate. A Hierarchical pattern was considered for complex compound requests (e.g., 'Prepare me for tomorrow'), but the Router pattern handles the majority of use cases cleanly and can be extended with sub-routing for compound requests in a future iteration.

Architecture Overview

The system is built in n8n and consists of one entry point, one routing layer, four specialist agent branches, a conditional event-creation sub-flow, and a unified Response Formatter. Every agent branch is independently testable and independently maintainable.

Data Flow

User Message (Chat Trigger)



Orchestrator Agent → Intent Classification (BRIEFING / SCHEDULING / EMAIL / FOLLOWUP / CLARIFY)



Intent Router (Switch Node) → 5 output branches

Branch 1: Google Calendar API (today + week) → Briefing Agent → Formatter

Branch 2: Google Calendar API (7-day window) → Scheduler Agent → Confirmation Check

IF EVENT_CREATE signal → Create Calendar Event (with Google Meet) → Formatter

ELSE → Formatter (clarification / confirmation message)

Branch 3: Gmail API (unread, last 24h) → Email Agent → Formatter

Branch 4: Gmail API (meeting subjects, 3d) → Follow-Up Agent → Formatter

Branch 5: Clarify Agent → Formatter

Agent Definitions

Agent	Single Responsibility	Input	Output	LLM / Tokens
Orchestrator Agent	Classify user intent into one of five routing keywords	Raw user chat message	Single uppercase string: BRIEFING, SCHEDULING, EMAIL, FOLLOWUP, or CLARIFY	GPT-4o-mini Temp 0 Max 20 tokens
Intent Router	Direct workflow execution to the correct specialist branch	Orchestrator output string	n8n Switch node: routes to one of five downstream branches	No LLM — pure logic node
Briefing Agent	Generate a structured daily summary from live calendar data	User request + Google Calendar events JSON (today and/or week)	Formatted briefing: meetings, conflicts, focus blocks, prep suggestions	GPT-4o-mini Temp 0.2 Max 800 tokens
Scheduler Agent	Handle all meeting scheduling requests with availability awareness	User request + existing calendar events for next 7 days	Confirmation summary, conflict warning, clarifying question, or EVENT_CREATE signal string	GPT-4o-mini Temp 0.1 Max 600 tokens
Email Agent	Triage and summarise the Gmail inbox	User request + Gmail message metadata (last 24h, unread)	Categorised list: Action Required vs FYI, plus inbox statistics	GPT-4.1-mini Temp 0.2 Max 700 tokens
Follow-Up Agent	Generate meeting summaries, action items, and draft follow-up emails	User request + pasted notes/transcript + recent meeting-related email context	Meeting summary, action item list with owners, draft follow-up email	GPT-4o-mini Temp 0.3 Max 900 tokens
Clarify Agent	Handle ambiguous requests with a single focused clarifying question	User chat message (unclear intent)	One friendly question presenting the four CalendarMate capabilities	GPT-4o-mini Temp 0.5 Max 150 tokens
Response Formatter	Normalise all agent outputs for consistent chat display	Output from whichever agent executed	Cleaned, formatted response string for the chat interface. Handles calendar creation confirmation specially.	n8n Code Node — no LLM

Tool Selection Rationale

Category	Choice	Rationale
Workflow Platform	n8n (self-hosted)	Visual multi-agent builder with native Google Calendar and Gmail nodes, built-in OAuth2 credential management, and direct LangChain integration via @n8n/n8n-nodes-langchain. No custom API wrappers required.
LLM (primary)	GPT-4o-mini	Cost-efficient at \$0.00015/1K input tokens and \$0.0006/1K output tokens, with reasoning sufficient for classification, summarisation, and structured generation. Used for 5 of 6 agent roles.
LLM (email)	GPT-4.1-mini	Used specifically for the Email Agent due to its strong instruction-following for categorisation tasks and slightly improved handling of multi-item structured output.
Calendar API	Google Calendar API v3 via OAuth2	Native n8n node with getAll (read), create (write), and availability querying. Required for all three core capabilities that involve calendar data.
Email API	Gmail API via OAuth2	Native n8n Gmail node supporting query-filtered message retrieval (is:unread, subject filters, date ranges) without requiring custom HTTP requests.
Authentication	Google OAuth2 (single credential)	One OAuth2 credential covers both Calendar and Gmail scopes, reducing setup complexity. Managed via n8n's built-in credential store — no secrets in workflow JSON.
Observability / Evaluation	Google Colab (Python)	All 12 baseline test inputs were run through the pipeline using a Python evaluation script in Google Colab. Each agent's output was captured, reviewed against expected criteria, and prompts were iteratively improved until all 12 tests passed. This provided full input/output visibility per agent across every test run.

Core Capabilities

Capability 1 — Daily Briefing

What it does: The Briefing Agent pulls all calendar events for the current day (Get Today's Events node) and the current week (Get Week Events node) via the Google Calendar API. These are passed alongside the user's request to GPT-4o-mini, which produces a structured output containing: a meeting list with readable times and attendees, a conflict detection section (any overlapping time slots), identified focus blocks (gaps of 30+ minutes), and one suggested prep action per external meeting.

Hallucination guardrail: The system prompt includes an explicit instruction: 'Use ONLY events from the JSON input. If the event JSON is empty or null, say Your calendar appears to be clear. Never invent any meetings, attendees, or times.' The temperature is set to 0.2 to minimise creative deviation from the source data.

Test inputs covered: T1 (today's summary), T2 (tomorrow's meetings), T10 (weekly conflict check), T12 (specific day lookup).

Capability 2 — Smart Scheduling

What it does: The Scheduler Agent receives the user's natural language scheduling request alongside a 7-day window of existing calendar events fetched by the Check Availability node. The agent follows a five-scenario decision tree: (A) confirm and prepare to book if all details are present and the slot is free; (B) name conflicts and propose alternatives if the slot is taken; (C) ask for the single missing piece of information (attendee, time, date, or duration — one question at a time); (D) flag unusual requests such as weekends or early/late hours; (E) emit a structured EVENT_CREATE signal string when the user explicitly confirms.

The EVENT_CREATE pattern: Rather than triggering event creation directly from the LLM, the Scheduler Agent outputs a pipe-delimited string in the format EVENT_CREATE:title|emails|startISO|endISO|description. A downstream Confirmation Check (IF node) detects this string and routes to the Create Calendar Event node, which parses the fields and calls the Google Calendar API to create the event with attendees and a Google Meet link. This two-step pattern prevents accidental event creation and makes the trigger explicit and fully auditable.

Hallucination guardrail: Never create or confirm an event if the attendee is missing. Never invent email addresses. Do not book a slot already occupied by another event in the JSON data. Default duration is 30 minutes if not specified. Working hours are 9:00 AM – 6:00 PM, Monday–Friday unless the user explicitly requests otherwise.

Test inputs covered: T3 (standard scheduling), T4 (ambiguous — must clarify), T5 (multi-attendee), T6 (weekend edge case), T9 (missing attendee), T11 (conflict detection).

Capability 3 — Email Summarisation

What it does: The Fetch Unread Emails node queries Gmail for up to 25 unread messages from the last 24 hours using the filter is:unread newer_than:1d. The metadata (From, Subject, Date, snippet) is passed to the Email Agent, which categorises each email as either Action Required (when subject or snippet contains keywords such as 'please review', 'urgent', 'approve', 'deadline', 'your input', or 'can you') or FYI (newsletters, no-reply senders, automated notifications, CC-only emails). The output includes a count of total unread emails and the most urgent item identified.

Hallucination guardrail: Only summarise emails present in the input JSON. If the inbox is empty, say 'No unread emails found in the last 24 hours.' Never quote full email bodies — one sentence per email maximum. Missing senders are noted as [unknown sender].

Test inputs covered: T7 (full inbox summary), T8 (attention-required emails only).

Extended Capability — Meeting Follow-Up Generator

What it does: The Fetch Meeting Emails node pulls up to 10 recent emails with meeting-related subjects (meeting, standup, sync, call, discussion) from the last 3 days as context. The Follow-Up Agent then processes the user's request — which must include pasted meeting notes or a transcript — and produces three outputs: a 2–3 sentence meeting summary, a structured action item list with owner names and due dates, and a complete draft follow-up email ready to send. The draft email includes a proper subject line, professional greeting, summary paragraph, numbered action items, and closing.

Hallucination guardrail: If the user provides no meeting notes or transcript, the agent responds with a specific prompt for notes rather than generating content from nothing. Action item owners are assigned

only to people explicitly named in the notes — never guessed. Decisions not stated in the input are never included. Due dates default to TBD if not mentioned.

Observability & Evaluation

Evaluation was conducted using a Python script run in Google Colab. Each of the 12 baseline test inputs was passed through the CalendarMate pipeline, and the full output of every agent was captured and reviewed against the expected criteria defined in the test dataset. This gave complete visibility into what each agent received as input and what it returned as output for every single test case.

How the evaluation loop worked: The 12 test inputs were run in sequence. For any test that failed or produced output that did not meet the expected criteria, the relevant agent's system prompt was updated to address the specific failure — a tighter guardrail, a clearer instruction, or a stricter output rule. The full set of 12 tests was then re-run from the beginning. This cycle repeated until all 12 tests passed with outputs that met every expected criterion and contained zero hallucinations.

Key finding — Orchestrator output formatting: During early evaluation runs, the Orchestrator Agent was observed wrapping its routing keyword in backticks — returning `BRIEFING` instead of BRIEFING — which caused the Intent Router Switch node to fail matching and fall through to the Clarify branch regardless of the user's intent. This was identified by reviewing the raw agent output captured in the Colab evaluation script. The system prompt was updated to explicitly prohibit markdown and backticks, and max tokens was reduced to 20 to physically prevent any output beyond the single keyword. After this fix, all routing tests passed correctly.

Key finding — Follow-Up Agent boundary case: Testing revealed that when a user message contained only a meeting name with no notes, the Follow-Up Agent sometimes drew on email context to construct a partial summary rather than asking for notes. The prompt was updated to require the presence of actual transcript or bullet-point notes before any generation occurs. Re-running the affected tests confirmed the fix.

Outcome: All 12 baseline test inputs passed after iterative prompt refinement. Zero hallucinations were detected across the full dataset. The evaluation process directly drove three prompt updates — to the Orchestrator, the Scheduler, and the Follow-Up Agent — each of which was verified by re-running the complete test suite.

Baseline Test Dataset Results

ID	Input	Type	Expected Criteria Met?	Hallucination?	Result
T 1	What does my day look like today?	Briefing	Listed today's meetings with times and attendees. Conflicts flagged.	None	PASS
T 2	Give me a summary of my meetings for tomorrow	Briefing	Tomorrow's meetings listed with times. No invented meetings.	None	PASS
T 3	Schedule a 30-min meeting with	Scheduling	Availability checked. Confirmation summary	None	PASS

I D	Input	Type	Expected Criteria Met?	Halluci nation?	Res ult
	sarah@company.com tomorrow at 2pm		with Meet link shown. Event created on confirmation.		
T 4	Set up a sync with the design team sometime this week	Scheduling (ambiguous)	Correctly asked: 'Who specifically from the design team?' and 'What day works best?' — no event created.	None	PAS S
T 5	Find a time for a 1-hour meeting with john@ext.com and lisa@ext.com next Monday	Scheduling (multi-attendee)	Checked availability window. Proposed 3 open slots rather than booking directly.	None	PAS S
T 6	Schedule something for Sunday at 6am	Scheduling (edge case)	Flagged as weekend and outside typical hours. Asked for confirmation before proceeding.	None	PAS S
T 7	Summarise my unread emails	Email	Grouped by Action Required / FYI. Included sender and subject per email.	None	PAS S
T 8	What emails need my attention today?	Email	Showed Action Required section only. Sender and subject included for each item.	None	PAS S
T 9	Book a meeting but I forgot with whom	Scheduling (missing info)	Responded: 'Who should I invite? Please share their email address.' No event created.	None	PAS S
T 10	Do I have any conflicts this week?	Briefing	Fetches full week events. Listed overlapping slots by name and time. Confirmed none when calendar was clear.	None	PAS S
T 11	Schedule a meeting at the same time as my existing standup	Scheduling (conflict)	Detected the conflict. Named the conflicting event. Offered 3 alternative slots.	None	PAS S
T 12	What's happening next Thursday?	Briefing	Returned correct day's events. Did not show wrong day's data.	None	PAS S

Baseline Result

All 12 test cases passed. Zero hallucinations detected across the full dataset. The critical edge cases (T4, T6, T9, T11) all triggered the correct clarification or warning behaviour rather than proceeding with incomplete or unsafe inputs.

Cost Analysis

Cost Item	Detail	Estimate
Tokens per Orchestrator run	~150 input (system prompt + user message) + 10 output	~160 tokens
Tokens per Briefing run	~800 input (prompt + calendar JSON) + 600 output	~1,400 tokens
Tokens per Scheduling run	~900 input (prompt + calendar JSON + user message) + 500 output	~1,400 tokens
Tokens per Email run	~1,200 input (prompt + email metadata × 25) + 600 output	~1,800 tokens
Tokens per Follow-Up run	~1,000 input (prompt + notes + email context) + 800 output	~1,800 tokens
Average tokens per pipeline run (all types)	Weighted average across all 5 branch types	~1,400 tokens
GPT-4o-mini pricing	Input \$0.00015/1K tokens Output \$0.0006/1K tokens	~\$0.00052 per average run
Estimated runs per user per day	1 briefing + 4 scheduling + 5 email + 2 follow-up + 3 misc	~15 runs/day
Cost per user per day	15 runs × \$0.00052	~\$0.0078/day
Cost per user per month	~\$0.0078 × 22 working days	~\$0.17/month
Cost for 50 users per month	50 × \$0.17	~\$8.50/month
Google Calendar + Gmail API	Within free quota for standard usage	\$0.00
Google Colab (Python evaluation)	Used for running all 12 baseline tests and iterative prompt refinement	\$0.00
Total estimated monthly cost (50 users)	LLM costs only — infrastructure via n8n Cloud or self-hosted	~\$8.50 – \$20/month

ROI summary: At under \$20/month for 50 users, CalendarMate costs approximately \$0.34 per user per month. If it recovers even 30 minutes of productive time daily per PM/TPM — a conservative target — the value generated is hundreds of times the cost.

Evaluation Strategy & Production Metrics

Three primary metrics will be tracked in production using a combination of output logging, periodic user surveys, and execution log analysis:

Metric 1 — Scheduling Accuracy Rate

Definition: Percentage of calendar events created by CalendarMate that match the user's confirmed parameters (correct time, correct attendees, no conflict).

Measurement: Cross-reference the parameters captured at the point of user confirmation (the EVENT_CREATE signal) against the actual calendar event created via the Google Calendar API. Flag any mismatches in time, attendees, or title.

Target: $\geq 95\%$. Anything below 90% triggers a prompt engineering review of the Scheduler Agent.

Metric 2 — Hallucination Rate

Definition: Percentage of AI outputs containing factual claims not traceable to source data (calendar events, email metadata, or user-provided notes).

Measurement: Monthly manual audit of a random sample of 50 outputs. Automated check: compare meeting titles and attendees in briefing outputs against the Calendar API response used as input for the same run.

Target: $< 2\%$ of pipeline runs. Any detected hallucination type is addressed immediately with a targeted prompt guardrail addition.

Metric 3 — Time Saved Per User Per Day

Definition: Self-reported minutes saved on meeting logistics (morning triage, scheduling, email management) compared to the pre-CalendarMate workflow.

Measurement: Bi-weekly pulse survey to all 50 users (3 questions, 2 minutes). Corroborated by n8n execution log data showing daily usage patterns and peak usage times.

Target: ≥ 60 minutes saved per user per day within 30 days of rollout. The pre-deployment baseline is 120 minutes per day on logistics.

Program Charter

Program Vision

Vision Statement

To eliminate the operational overhead of meeting logistics for ServionIQ's PM and TPM community — giving every product and technical leader back the time and cognitive space to focus on the high-value work that drives product outcomes, not the coordination tasks that surround them.

CalendarMate represents ServionIQ's first production deployment of Agentic AI for internal productivity. Its success will establish the architectural and organisational patterns for all subsequent AI assistant deployments across the company.

Scope

In Scope — Phase 1

- Daily briefing generation from Google Calendar for the current and upcoming day
- Natural language meeting scheduling with availability checking and Google Meet link generation
- Gmail inbox triage — unread email categorisation into Action Required and FYI
- Meeting follow-up generation from user-provided notes: summary, action items, and draft email
- Deployment for internal ServionIQ PMs and TPMs (target: 50 users)
- Evaluation via Python scripts in Google Colab — all 12 baseline tests run and documented with iterative prompt refinement

Out of Scope — Phase 1

- External client scheduling or client-facing calendar integrations
- Slack, Microsoft Teams, or Zoom integration (Phase 2)
- Microsoft Outlook / Office 365 calendar support (Phase 2)
- Automated email sending without human review (human-in-the-loop retained for all outbound emails)
- Fine-tuning of custom models (optional enhancement — Phase 2 if Phase 1 metrics are met)
- Multi-language support

Success Criteria

Success Criterion	Measurement	Target	Timeline
Time savings	User survey: avg. minutes/day saved on meeting logistics	≥ 60 min/user/day	30 days post-launch
Hallucination rate	Monthly trace audit: % of outputs with invented data	< 2%	Ongoing from Day 1

Success Criterion	Measurement	Target	Timeline
Scheduling accuracy	% of created events matching confirmed parameters	≥ 95%	Ongoing from Day 1
User adoption	% of target PMs/TPMs with at least 1 daily interaction	≥ 80% DAU	60 days post-launch
System reliability	% of pipeline runs completing without error	≥ 99% uptime	Ongoing from Day 1
Cost efficiency	Total LLM cost per user per month	< \$0.50/user/month	Ongoing from Day 1

Timeline

Phase	Duration	Key Activities	Milestone
Phase 0: Foundation	Week 1–2	Google OAuth2 setup, n8n workflow build, Python evaluation in Google Colab, all 12 baseline tests passing with iterative prompt refinement	CalendarMate v1.0 ready for pilot
Phase 1: Pilot	Week 3–4	Deploy to 5 volunteer PMs/TPMs. Collect feedback. Review outputs and logs. Fix prompt issues identified in real-world usage.	Pilot feedback report. Prompt v2 deployed.
Phase 2: Staged Rollout	Week 5–6	Expand to 25 users. Monitor metrics dashboard daily. Conduct weekly office hours for questions and edge cases.	25-user cohort active. First time-savings survey.
Phase 3: Full Rollout	Week 7–8	Deploy to all 50 users. Launch internal documentation and video walkthrough. Establish monthly metric review cadence.	50 users active. Program charter KPIs in review.
Phase 4: Optimise	Week 9–12	Analyse 30-day data. Identify top hallucination patterns. Update prompts. Evaluate fine-tuning candidates. Plan Phase 2 features.	Phase 2 feature backlog prioritised.

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
LLM hallucination in briefings or scheduling outputs erodes user trust	Medium	High	Strict grounding rules in all system prompts. Hallucination rate monitored via periodic manual output audits. Human confirmation required before any event creation.
Google OAuth2 setup complexity	Medium	Medium	Provide step-by-step OAuth2 setup guide with screenshots. Offer 30-minute

Risk	Likelihood	Impact	Mitigation
blocks adoption for non-technical users			onboarding session per user cohort. n8n credential store handles token refresh automatically.
Users do not trust AI to manage their calendar and default to old workflow	Medium	High	Human-in-the-loop design: CalendarMate never creates events or sends emails without explicit user confirmation. Pilot with 5 enthusiastic early adopters to build internal advocates.
Token costs exceed projections if usage is higher than estimated	Low	Low	Cost monitoring via OpenAI usage dashboard with monthly budget alerts. GPT-4o-mini is already the cheapest capable model. Cost ceiling remains < \$1/user/month even at 3× estimated usage.
Orchestrator misroutes ambiguous requests, degrading experience	Low	Medium	Clarify Agent handles all unmatched intents gracefully. Orchestrator prompt has been tested against 12 baseline inputs and all edge cases with zero misroutes after prompt v2.
Gmail API rate limits hit during peak usage (50 users, morning rush)	Low	Medium	Stagger briefing triggers via n8n Schedule Trigger rather than all users requesting simultaneously. Monitor Gmail API quota in Google Cloud Console.

Stakeholders

Stakeholder	Role	Interest	Engagement Model
CTO	Executive Sponsor	Strategic AI adoption, ROI, and organisational capability building	Monthly steering update. Final sign-off on Phase 2 scope.
PM / TPM Team (50 users)	Primary Users	Time savings, reliability, and ease of use	Onboarding sessions, bi-weekly pulse surveys, dedicated Slack channel for feedback.
Engineering Lead	Technical Reviewer	System architecture, API security, data handling, and n8n infrastructure	Design review at Phase 0. Monthly infrastructure review.
Legal / Compliance	Risk Advisor	Data privacy for calendar and email processing, OAuth2 scope minimisation	Review at Phase 0 before any OAuth2 deployment. Sign-off on data handling policy.
IT / DevOps	Infrastructure Owner	n8n deployment, uptime, credential management, and Google Workspace integration	Involved in Phase 0 setup. On-call support during rollout phases.

Decision-Making Framework

Prompt changes: Any modification to an agent's system prompt requires re-running the full 12-test baseline dataset and documenting before/after outputs before deployment.

Model upgrades: Upgrading any agent's LLM model requires a cost impact analysis and baseline re-test. Approval from the Engineering Lead required if monthly cost impact exceeds \$50.

New feature additions: All new capabilities must be scoped with a problem statement, success metric, and test dataset of at least 5 inputs before build begins.

Rollback trigger: If the hallucination rate exceeds 5% in any 7-day period, or if system uptime drops below 95%, an immediate rollback to the previous workflow version is initiated while root cause analysis is performed.

Rollout Plan for 50 PMs/TPMs

1. Identify 5 volunteer pilot users from the PM/TPM team — preferably those with the heaviest meeting loads who will see the most immediate benefit.
2. Conduct a 45-minute onboarding session per cohort covering: what CalendarMate does, what it does not do, how to write effective natural language requests, and how to report issues.
3. Deploy Google OAuth2 credentials for each user via the IT team. Provide a written setup guide with screenshots for self-service troubleshooting.
4. Monitor agent outputs closely during the pilot phase. Run the baseline test suite periodically against the live system and review outputs for any new failure patterns introduced by real-world input variation.
5. After each phase, distribute a 3-question pulse survey: (1) How many minutes per day did CalendarMate save you? (2) Did you encounter any incorrect outputs? (3) What capability would you most like added?
6. Use survey responses and trace data to prioritise prompt improvements before each phase expansion.
7. Publish a monthly internal metrics report to all users showing time saved, accuracy rates, and cost — building transparency and trust in the system.

Reflection & Next Steps

Evaluation Findings & What They Revealed

Evaluation was conducted by running all 12 baseline test inputs through the CalendarMate pipeline using a Python script in Google Colab. Each agent's output was captured and reviewed against the expected criteria. Where outputs failed or contained issues, the relevant system prompt was updated and the full test suite was re-run from scratch. This cycle continued until every test passed cleanly. Three specific findings shaped the final system design.

Finding 1 — Orchestrator Markdown Leakage

When the Orchestrator Agent's output was printed in the Colab evaluation script, it was observed occasionally wrapping its routing keyword in backticks — outputting ``BRIEFING`` instead of *BRIEFING*. This caused the Intent Router's Switch node to fail matching, and the workflow fell through to the default Clarify branch regardless of what the user asked. The root cause was that GPT-4o-mini, when prompted without a hard output constraint, applies its default formatting conventions to short text outputs.

Fix applied: Two changes were made simultaneously. First, the system prompt was updated to add: 'No markdown. No backticks. No explanation. No punctuation. Just the single routing keyword.' Second, the max tokens parameter on the Orchestrator LLM was reduced from 50 to 20, physically preventing the model from producing anything longer than the keyword itself. After this change, all 12 baseline tests routed correctly with zero failures.

Finding 2 — Scheduler Agent on Dense Calendar Data

When evaluation inputs included a large number of calendar events in the 7-day window, the Scheduler Agent's outputs remained accurate and correctly grounded — no hallucinated slots or invented attendees. However, reviewing the outputs in Colab confirmed that the agent was processing significantly more input data than necessary. For a request about a single day, the full 7-day calendar JSON was being passed in. The outputs were correct, but the efficiency could be improved.

Optimisation identified for future: A pre-processing step to extract only events for the requested day plus the adjacent day would reduce input data by approximately 50% on typical scheduling requests, cutting both token usage and response time.

Finding 3 — Follow-Up Agent Robustness

Testing the Follow-Up Agent with no meeting notes confirmed it correctly refuses to generate content — it responds with the request for notes and produces nothing else. However, when the evaluation input contained only a meeting name with no actual notes, the agent sometimes used the email context data to attempt a partial summary. This was caught by reviewing the Colab output for that specific test case. The guardrail at that point only checked for the absence of notes but did not explicitly reject a meeting-name-only input.

Fix applied: The system prompt was updated to add an explicit check: 'If the user's message does not contain a transcript or bullet-point notes, respond with the notes-request message. The presence of a meeting name alone is insufficient to generate content.'

Two-Week Improvement Plan

Week	Priority	Action	Success Measure
Week 1	HIGH	Implement calendar data pre-filtering: extract only events for the requested day (+ adjacent day) before passing to Scheduler Agent, reducing average input data by ~50%.	Scheduling branch outputs remain accurate and response time visibly improves on re-evaluation in Colab.
Week 1	HIGH	Add a Conflict Resolver as an extended capability: when two events overlap in the Briefing output, automatically suggest rescheduling options for the lower-priority one.	T10 and T11 produce not just conflict detection but actionable reschedule proposals.
Week 1	MEDIUM	Run fine-tuning evaluation for the Orchestrator Agent: create 50 JSONL training examples (40 clear-intent, 10 ambiguous) to reduce the rate of CLARIFY on borderline inputs.	CLARIFY rate on clear-intent inputs drops to < 5% when re-run through the evaluation script.
Week 2	HIGH	Add a Weekly Digest capability: triggered every Friday at 4 PM via n8n Schedule Trigger, generates a summary of the week's meetings, action items, email volume, and focus time.	Weekly digest output tested against 5 sample inputs and passes hallucination check.
Week 2	MEDIUM	Implement Attendee Research as an extended capability: before a meeting with an external attendee, the Briefing Agent retrieves and summarises the most recent email thread with that person.	Tested with 3 real external attendee scenarios. Output is grounded in actual email content.
Week 2	MEDIUM	Connect a human-in-the-loop approval step for outbound follow-up emails: before the draft follow-up is sent, route it to the user for review via a Slack message or email preview.	User sees and approves or edits the draft before it is sent. Zero automated sends without approval.

Privacy & Security Considerations

Data Minimisation

CalendarMate accesses the minimum data necessary for each capability. The Email Agent retrieves only message metadata (From, Subject, Date, snippet) — never full email bodies — unless the user explicitly pastes content into the chat. The Briefing Agent retrieves calendar event data only for the relevant time window (today or this week), not the full calendar history. The Follow-Up Agent processes notes only within the active session; nothing is stored persistently.

OAuth2 Credential Security

Google OAuth2 tokens are stored exclusively in n8n's encrypted credential store. They are never embedded in workflow JSON or logged to any external system. Token refresh is handled automatically.

by n8n. Credentials should be scoped to the minimum required OAuth2 scopes: calendar.events (read/write) and gmail.readonly (for inbox reads), with gmail.send added only if automated follow-up sending is enabled in a future phase.

Evaluation Data Handling

The Python evaluation scripts run in Google Colab process agent inputs and outputs only within the active session. No calendar event details or email metadata are stored to disk or shared externally during evaluation runs. When this approach is extended to production monitoring, the same principle applies — output samples reviewed for quality should be anonymised before any storage and retained only for the minimum period needed to support prompt improvement decisions.

Human-in-the-Loop as a Privacy Safeguard

The most significant privacy and trust risk in a calendar/email AI assistant is unsanctioned actions — creating events the user did not intend, or sending emails without review. CalendarMate's design requires explicit user confirmation before any write action is executed. The EVENT_CREATE signal is only emitted when the user types a clear confirmation. This is not just a UX choice — it is the primary safeguard against the assistant taking irreversible actions based on misunderstood input.

Rollout Plan for 50 PMs/TPMs

Phase 1: Pilot (Users 1–5, Weeks 1–2)

Select five pilot users based on two criteria: they have the heaviest meeting loads (most to gain), and they are comfortable giving structured feedback (most useful signal). Run CalendarMate alongside their existing workflow — not as a replacement. Collect feedback through a daily 2-minute Slack check-in and a formal feedback session at the end of Week 2.

Phase 2: Staged Expansion (Users 6–25, Weeks 3–4)

Expand to 25 users after incorporating pilot feedback and deploying Prompt v2. Conduct a group onboarding session (45 minutes) covering: what CalendarMate does well, what it does not do, and how to phrase requests for best results. Run the baseline evaluation suite periodically against the live system to check for new failure patterns introduced by real-world input variation.

Phase 3: Full Rollout (Users 26–50, Weeks 5–6)

Deploy to the full team of 50 PMs/TPMs. Publish internal documentation: a one-page quick-start guide and a 5-minute video walkthrough. Establish a #calendarmate-feedback Slack channel for real-time issue reporting. Issue the first formal time-savings survey at 30 days post-launch.

Phase 4: Optimise & Plan Phase 2 (Weeks 7–12)

Review 30-day metric data against success criteria. If hallucination rate is below 2% and time savings exceed 60 minutes/day, proceed with Phase 2 scoping: Slack integration, Microsoft 365 support, and fine-tuned Briefing Agent. If metrics are not met, identify root causes from output reviews and re-run the evaluation suite to address with targeted prompt updates before expanding scope.

Evaluation & Fine-Tuning Approach

Evaluation Loop Applied

The evaluation-optimisation loop from the course was directly applied to CalendarMate during development using a Python script in Google Colab. The specific cycle executed was:

8. Run all 12 baseline test inputs through the pipeline and capture each agent's output
9. Review outputs against expected criteria — identify the Orchestrator backtick formatting failure
10. Update the Orchestrator system prompt and reduce max tokens to 20
11. Re-run the full 12 tests and confirm zero routing failures
12. Identify the Follow-Up Agent boundary case from output review — update the prompt
13. Re-run all 12 tests again and confirm all pass with zero hallucinations

This loop — running tests, capturing outputs, identifying failures, fixing prompts, and re-running to verify — is what distinguishes a production-grade system from a prototype that passes a demo. The value of running tests in a structured evaluation script is that failures are visible and documented, not discovered by chance in live usage.

Fine-Tuning Strategy

Best candidate: The Orchestrator Agent is the strongest candidate for fine-tuning, because it is the highest-leverage node in the system — a routing error at this point causes the entire pipeline to fail regardless of how well the specialist agents perform. Fine-tuning would target borderline inputs that currently trigger CLARIFY when a reasonable human would route them to a specific capability.

Training data approach: Generate 50 JSONL training examples in system/user/assistant format using Claude or GPT-4o to create diverse inputs. The dataset should include: 15 clear BRIEFING examples, 15 clear SCHEDULING examples, 10 EMAIL examples, 5 FOLLOWUP examples, and 5 genuinely ambiguous CLARIFY examples. The goal is to push the CLARIFY rate on borderline-but-classifiable inputs toward zero while maintaining correct CLARIFY behaviour on truly ambiguous messages.

Measurement: Run the full 12-test baseline dataset before and after fine-tuning using the same Python evaluation script in Google Colab. Track the CLARIFY rate specifically on T4 (ambiguous scheduling) and compare Orchestrator response quality before and after. A fine-tuned model should be faster and more consistent on the Orchestrator task because it needs less in-context instruction to behave correctly.

Reflection Summary

CalendarMate demonstrates that the most important skills in building production agentic AI are not architectural sophistication but disciplined prompt engineering, systematic evaluation, and honest documentation of failures and fixes. Running all 12 test cases through a Python evaluation script in Google Colab and iterating prompts until every test passed — that process is what makes this system production-ready. Every agent knows exactly what to do when it does not have enough information, and that behaviour was tested, broken, fixed, and verified before deployment.